



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Module Objective: In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```

CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set           // To store unique string fact
    DECLARE rules AS List         // To store rules as Tuples: (

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules
  
```

```
FUNCTION clear_facts():  
    EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():  
    DECLARE new_facts_added = TRUE  
  
    WHILE new_facts_added == TRUE:  
        new_facts_added = FALSE  
  
        FOR EACH (premises, conclusion) IN rules:  
            IF conclusion IS NOT IN facts:  
  
                // Modus Ponens Check  
                IF ALL premises ARE IN facts:  
                    ADD conclusion TO facts  
                    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py`, instantiate the KB in the `__init__` method: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
Rule 1: `TargetVisible  $\wedge$  HasDust  $\Rightarrow$  SafeToEngage`
Rule 2: `SafeToEngage  $\wedge$  BloodseekerMissing  $\Rightarrow$  Retreat`

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list`, clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()`.
4. Run `self.kb.forward_chain()`.
5. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```

from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce Sa
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()

```

Part 5: Theory-to-Implementation Mapping

Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your

new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

In Step 3.2, we modified the A* algorithm's neighbor-generation loop to distinguish between reachability and feasibility. Reachability is the physical check that determines whether a tile can be moved to based on grid boundaries and wall collisions, which is the standard A* functionality. Feasibility, however, is the logical check that determines whether a tile is safe to enter based on the knowledge base's inference results. The modification works by first checking if a neighbor tile is physically reachable, then clearing the KB facts, feeding the current percepts for that specific tile as facts, running `forward_chain()` to deduce any new information, and finally checking if 'Retreat' exists in the facts. If 'Retreat' is deduced, the tile is marked as infeasible and skipped from the open list, even though it is physically reachable. This creates a two-tier validation system where logical reasoning overrides pure geometric pathfinding, demonstrating that a tile might be physically adjacent but logically forbidden due to safety rules.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts: directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?`

We built a `KnowledgeBase` class with a `forward_chain()` loop instead of writing procedural if-else statements directly in the agent's movement code because this represents the declarative programming paradigm, which offers significant advantages over procedural approaches. With a declarative knowledge base, rules become data rather than code, achieving separation of concerns where the movement logic remains completely unchanged regardless of how many rules are added. If we add 50 new game rules, the procedural approach would require modifying the agent's movement code by adding 50 new if-else conditions, introducing high risk of bugs and making the code increasingly complex and difficult to maintain. In contrast, with the declarative Knowledge Base approach, adding 50 new rules only requires 50 additional `tell_rule()` calls without touching the movement logic at all, maintaining $O(n)$ complexity and keeping the code clean and maintainable. Furthermore, rules can be added, removed, or modified without affecting the A* algorithm, multiple agents can share the same rule base, and domain knowledge is stored separately from programming logic, making debugging easier by isolating rule issues from movement issues.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause, which is a logical rule of the form $(\text{Premise}_1 \wedge \text{Premise}_2 \wedge \dots \wedge \text{Premise}_n) \rightarrow \text{Conclusion}$. The `forward_chain()` `while` loop acts as a mathematical execution of the Modus Ponens inference rule by repeatedly applying the principle that if we have a rule $P \rightarrow Q$ and P is true, then we can deduce Q . In the implementation, Modus Ponens is executed when the `forward_chain()` method iterates through each rule, first checking if the conclusion is not already in the facts set, then using `all(premise in self.facts for premise in premises)` to verify that all premises exist in the current facts, and finally adding the conclusion to the facts set with `self.facts.add(conclusion)`. The `while` loop continues this process until a full pass through all rules results in no new facts being added, which mathematically computes the least fixpoint of the Horn clause program and ensures that all facts that logically follow from the initial percepts are derived. For example, when the facts contain `TargetVisible` and `HasDust`, the first rule fires to add `SafeToEngage`, and if `BloodseekerMissing` is also present, the second rule then fires to add `Retreat`, demonstrating how Modus Ponens chains rules together to derive increasingly complex logical conclusions from simple initial facts.

Finalize and Lock Lab 05 Implementation



FACULTY OF COMPUTING